TEAM WORK 0 - Team 4 (UDC)

- Alejandro Silva
- Fernando Baña
- Daniel Prieto

1. Modification of the work areas

The main objective of this modification is to make the work areas so that Area 1 (upper) contains three welding joints (1 to 3) and Area 2 (lower) contains the other two weling joints (4 and 5) instead of having the previous distributions with left and right areas.

The main change is in the `JOINT_POS` array, where we have changed the positions of joints 3 and 4, and the creation `JOINT_AREA` array to map which joints belong to which area. The position of the joints has not been changed, even thought for the welding agent it is not relevant, it is important for the visualization of the factory.

```
    public static final int[][] JOINT_POS    =
        {
            {914,194}, // Joint 1 – Area 1 (upper-right)
            {501,197}, // Joint 2 – Area 1 (upper-center)
            {501,215}, // Joint 3 – Area 2 (upper-center)
            {534,460}, // Joint 4 – Area 1 (lower-center)
            {358,459}  // Joint 5 – Area 2 (lower-right)
        };

    // Map which joints belong to which area
    public static final int[] JOINT_AREA = { 1, 1, 1, 2, 2 };
```

In order to connect the logic from the welding agent to the new areas, we modified the `weldingagent.asl` in various places to use the new are mapping. We introduced new beliefs to keep track of which area is needed for each joint and the locked area. Reflecting the new distribution of joints in areas defined in the `FactoryArtifact`.

```
jointInArea(1, 1).
jointInArea(2, 1).
jointInArea(3, 1).
jointInArea(4, 2).
jointInArea(5, 2).
```

We have also modified the plans of the welding agent to request the correct area for each joint, not neccessarily the logic itself, but the way it interacts with the assembly area agent to request the area for welding. The main change is that we have grouped the plans into two plans, one for requesting the area and another for performing the weld.

```
// Plan to request the area for welding a joint
+!weldParts : jointPartsInPlace(Joint) & not joint(Joint) &
jointInArea(Joint, A) & not lockedArea(A)
   <- .print("Welding robot: requesting area ", A, " for joint ", Joint,
".");
      .my_name(Agent);
      .send(assemblyareaagent, achieve, lockAreaFor(Agent, A));
      .wait(1000);
      !weldParts.

// Plan to perform the welding of a joint when the area is locked
+!weldParts : jointPartsInPlace(Joint) & not joint(Joint) &
jointInArea(Joint, A) & lockedArea(A)
   <- .print("Welding robot: welding joint ", Joint, " in area ", A, ".");
      .drop_intention(parkArm);
      ?jointPos(Joint, X, Y);
      !moveTo(X, Y);
      weld;                          // CArtAgO operation
      +joint(Joint);
      .broadcast(tell, joint(Joint));
      !!parkArm;
      !weldParts.
```

These changes allow the welding agent to interact correctly with the new distributions of joints in the areas. However the rest of the agents, specifically the roboticarm agent was not modified, so it will not take into account the new distribution of the areas to place the parts in the holders.

2. Mounting bycicles constantly

The main objective of this modification is to make the factory work in a continuous way. The main problem spotted were the bin agents, which would only produce one part and then stop producing. In the beginning the refill plan was triggered by the belief deletion `-binfull(N)`, but this rule was derived from CArtAgO properties, and we think it was not working as expected because it was never being triggered reliably. To fix this, we removed the reactive `-binfull(N)` and replaced it with a monitoring loop. The agent will gets assigned a binm and loops forever checking the state of the bin:

- If the bin is full, it will wait for a while and check again until it becomes empty.
- If the bin is not full, it will wait for a random time and then refill the bin, and then check again.

```
// Fallback if the agents has not being assigned a bin yet.
+!refill : not binnumber(_)
<- .print("Bin agent: no bin number assigned, cannot refill.");
   .wait(1000);
   !refill.

// bin became full — wait for it to be emptied
+!refill : binnumber(N) & binfull(N)
```

```
   <- .wait(1000);
      !refill.

// OLD PART: only change is the addition of the last line to loop again
after refilling
+!refill : binnumber(N) & timer(T) & not binfull(N)
<- // math.random used as arithmetic expression (not as action)
   // to avoid CArtAgO intercepting it as an artifact operation.
   WaitTime = math.random * T;
   .print("Bin agent ", N, " waiting ", WaitTime div 1000, " s for new
parts...");
   .wait(WaitTime);
   .print("Bin agent ", N, " has received new parts.");
   refill_bin(N);        // CArtAgO operation on factory_env
   !refill.
```

With this modification, every time a part is consumed from the bin, the agent will trigger the refill plan. These changes make the factory work in a continous way.

3. How to obtain different artifacts for each agent

The main objective of this modification is to transition the system environment from a single artifact `FactoryArtifact` into specialized artifacts for each agent.

Intead of having all agents focus a single artifact that handles all the logic, we splitted the responsibilities so that each agent only interacts with the its own artifact.

The original `FactoryArtifact` was splitted into the following artifacts:

1. **`ArmArtifact.java`**: For to the `roboticarmagent`. It has the `pick_part`, `release_part`, and arm operations. It exposes the properties specific to the gripper position and content.
2. **`WelderArtifact.java`**: For to the `weldingagent`. It handles the `weld` and welder movement, keeping track of the welder's position.
3. **`MoverArtifact.java`**: For to the `movingagent`. It manages the movement of the finished frames and shares the mover's position.
4. **`AssemblyBoardArtifact.java`**: Acts as a shared environment artifact. It manages the assembly areas, holding states, and bin capacities. All agents will still interact with this shared space to coordinate locks and part placements.

To ensure the the agents interact with the correct artifacts, the agents initialization was updated. So instead of relying on the `lookupArtifact("factory_env", ArtId)`, each agent looks up its own specific artifact during the start, for instance the `weldingagent` looks up the `WelderArtifact`:

```
+!main : true
<- !focus_factory;
   makeArtifact("welder_tool", "factory.WelderArtifact", [], WelderId);
   focus(WelderId);
   +welder_art_id(WelderId);
```

```
    .print("Welding robot: waiting for new parts");
    !weldParts.
```

This leverage the separation of concerns that CArtAgO and Java provides. The system is more modular, and easier to maintain.